

# Comparativa de implementacion MQTT

Arduino / C++ vs Rust (esp-idf-svc)  
ESP32-S3 — Proyecto GiiRob PR2-A1

Este documento compara la implementacion MQTT en Arduino/C++ (biblioteca PubSubClient, archivos *e\_mqtt\_lib\_no\_tocar.ino* y *g\_comunicaciones.ino*) con la implementacion equivalente en Rust (*mqtt\_manager.rs*), mostrando fragmentos reales de ambos proyectos y explicando por que la version Rust es mas adecuada para la arquitectura de doble nucleo del ESP32-S3.

## 1. Modelo de ejecucion

Arduino ejecuta todo en un unico hilo. El main loop llama a `mqtt_loop()` en cada iteracion, que a su vez verifica la conexion, reconecta si es necesario y procesa un mensaje. El callback MQTT y la logica del robot comparten el mismo hilo de ejecucion.

Rust separa el callback MQTT (Core 0, Wi-Fi stack de FreeRTOS) de la logica del robot (Core 1, `logic_task`). El callback solo encola eventos a traves de un canal `mpsc::SyncSender`; nunca procesa directamente.

### Loop principal y recepcion de mensajes

En Arduino, `mqtt_loop()` debe llamarse en cada iteracion. En Rust, el callback es async y nunca bloquea.

#### Arduino / C++

*w\_loop.ino + e\_mqtt\_lib\_no\_tocar.ino*

```
// Cada iteracion del loop principal:
void on_loop() {
    mqtt_loop(); // llama en cada ciclo
}

void mqtt_loop() {
    if (!mqttClient.connected())
        mqtt_reconnect(3); // BLOQUEA hasta 15s
    mqttClient.loop(); // 1 mensaje por llamada
}

// Callback en mismo hilo que el loop:
void mqttCallback(char* topic,
    byte* message, unsigned int length) {
    String msg;
    for (int i=0; i<length; i++)
        msg += (char)message[i]; // copia byte a byte
    alRecibirMensajePorTopic(topic, msg);
}
```

#### Rust

*mqtt\_manager.rs*

```
// Core 0: callback MQTT (nunca bloquea)
let mut client = EspMqttClient::new_cb(
    config::MQTT_URL, &cfg,
    move |event| match event.payload() {
        EventPayload::Received {
            topic, data, ..
        } => {
            // Zero-copy: apunta al buffer del broker

            let msg = std::str::from_utf8(data)
                .unwrap_or("");
            // Solo encola, NO procesa
            event_tx.try_send(
                RobotEvent::DeltaCompleted {
                    color, id_cap
                }
            );
        }
    }
)?;
```

**Ventaja Rust:** El callback de Rust corre en Core 0 y solo encola el evento. Core 1 (`logic_task`) consume la cola sin bloquear la recepcion MQTT. En Arduino, si el callback tarda, todo el programa se detiene.

## 2. Reconexion al broker

La estrategia de reconexion es critica en sistemas industriales. Arduino usa reintentos fijos con `delay()` bloqueante; Rust reintenta indefinidamente en un hilo separado sin afectar el resto del sistema.

### Estrategia de reconexion y suscripcion

Arduino bloquea el programa completo durante la reconexion. Rust suspende solo el hilo de inicializacion.

Arduino / C++

*e\_mqtt\_lib\_no\_tocar.ino*

```
#define MQTT_CONNECTION_RETRIES 3

void mqtt_reconnect(int retries) {
    int r = 0;
    while (!mqttClient.connected()
        && r < retries) {
        r++;
        if (mqttClient.connect(
            mqttClientId.c_str())) {
            delay(1000); // bloquea 1s
        } else {
            delay(5000); // bloquea 5s
            // maximo 3 intentos, luego abandona
        }
    }
}

// Total posible bloqueado: 15 segundos
// Despues de 3 fallos: NO reintenta mas
```

Rust

*mqtt\_manager.rs*

```
fn subscribe_all_topics(
    client: &mut; EspMqttClient<'_,>,
    topics: &[&str;]
) {
    for &topic; in topics {
        loop { // reintenta INFINITAMENTE
            match client.subscribe(
                topic, QoS::AtLeastOnce) {
                Ok(_) => {
                    info!("Suscrito a {}", topic);
                    break;
                }
                Err(e) => {
                    error!("Reintentando...");
                    // solo suspende ESTE hilo
                    thread::sleep(
                        Duration::from_secs(2));
                }
            }
        }
    }
}
```

**Ventaja Rust:** Rust reintenta hasta tener exito, sin limite de intentos. `thread::sleep()` suspende unicamente el hilo de inicializacion; la logica del robot sigue corriendo en Core 1.

## 3. Estado compartido y seguridad en concurrencia

El estado del sistema (modo, contadores, flags de emergencia) debe compartirse entre el callback MQTT y la logica del robot. Arduino usa variables globales sin proteccion. Rust garantiza acceso seguro a traves del sistema de ownership y el compilador rechaza cualquier acceso sin mutex.

### Acceso al estado desde el callback

Variables globales en Arduino vs `Arc` en Rust. El compilador de Rust hace imposible el data race.

Arduino / C++

*e\_mqtt\_lib\_no\_tocar.ino + g\_comunicaciones.ino*

```
// Estado global, sin proteccion:
PubSubClient mqttClient(espWifiClient);
```

Rust

*mqtt\_manager.rs*

```
// Estado protegido por Mutex, compartido
// con Arc (reference counting atomico):
```

```
String mqttClientID;
// Cualquier funcion puede modificarlo
// sin ningun control de acceso

void alRecibirMensajePorTopic(
    char* topic, String msg) {
    // Acceso directo a variables globales
    if (strcmp(topic, HELLO_TOPIC)==0) {
        if(msg == "on") {
            // sin mutex, sin verificacion
            digitalWrite(LED, HIGH);
        }
    }
}
// En FreeRTOS con 2 tareas:
// DATA RACE -> comportamiento indefinido
```

```
pub fn connect_and_subscribe_with_state(
    control_state: Arc<Mutex<ControlState>>,
    emergency_stop: Arc<AtomicBool>,
    ...
) -> Result<Self> {

    // En el callback:
    if cmd == "set_mode" {
        // try_lock: NO bloquea si esta ocupado
        if let Ok(mut state) =
            control_state.try_lock() {
            state.mode = Mode::Auto;
        } else {
            error!("No se pudo lockear");
        }
    }
}
// Sin mutex -> ERROR DE COMPILACION
```

**Ventaja Rust:** Arc garantiza que solo un hilo modifica el estado a la vez. try\_lock() evita que el callback bloquee el Wi-Fi stack si logic\_task ya tiene el mutex. El compilador rechaza acceso sin mutex en tiempo de compilacion.

## 4. Senales criticas: parada de emergencia

La parada de emergencia requiere propagacion instantanea entre nucleos. Arduino no tiene mecanismo nativo para esto. Rust usa AtomicBool, una operacion atomica que no necesita mutex y es segura entre nucleos por hardware.

### Propagacion de emergencia entre nucleos

Arduino no tiene primitive atomica. Rust usa AtomicBool: escritura/lectura sin mutex, segura en hardware.

Arduino / C++

Rust

*g\_comunicaciones.ino (sin soporte real)*

*mqtt\_manager.rs*

```
// No existe primitive atomica en Arduino
// Se necesitaria una variable global:
bool emergencia = false; // NO thread-safe

void alRecibirMensajePorTopic(
    char* topic, String msg) {
    if (strcmp(topic, EMERGENCY)==0) {
        emergencia = true;
        // Otra tarea podria leer un valor
        // a medio escribir -> undefined behavior
    }
}
// En otra tarea FreeRTOS:
if (emergencia) { /* ... */ }
// Sin garantia de visibilidad entre cores
```

```
// AtomicBool: operacion atomica a nivel CPU
// No necesita mutex, segura entre nucleos
pub fn connect_and_subscribe_with_state(
    emergency_stop: Arc<AtomicBool>,
    ...
) {
    // Core 0: callback escribe atomicamente
    if cmd == "estop" {
        emergency_stop.store(
            true, Ordering::SeqCst
        );
    }
}
// Core 1: logic_task lee atomicamente
if emergency_stop.load(
    Ordering::SeqCst) {
```

```
// parada garantizada e inmediata
}
}
```

**Ventaja Rust:** Ordering::SeqCst garantiza que la escritura en Core 0 sea visible inmediatamente en Core 1. No hay ventana de tiempo donde el valor sea inconsistente. Esto es imposible de garantizar con bool global en Arduino.

## 5. Despacho de topics recibidos

Ambas implementaciones necesitan ejecutar logica distinta segun el topic MQTT recibido. Arduino usa strcmp encadenado (C-style). Rust usa match, que es exhaustivo: si se agrega un topic nuevo sin manejarlo, el compilador advierte.

### Enrutamiento por topic

strcmp encadenado vs match exhaustivo. Rust detecta en compilacion topics sin handler.

Arduino / C++

Rust

*g\_comunicaciones.ino*

```
void alRecibirMensajePorTopic(
    char* topic, String msg) {
    // strcmp: comparacion C-string manual
    if (strcmp(topic, HELLO_TOPIC)==0) {
        // ...manejar hello
    }
    // Si agregas un topic y olvidas
    // el if: falla en runtime, silenciosamente
    //
    // No hay garantia de cobertura completa
}
```

*mqtt\_manager.rs*

```
match topic_recibido {
    config::MQTT_TOPIC_SCADA_ACTION => {
        // ...manejar SCADA
    }
    config::MQTT_TOPIC_DELTA_STATUS => {
        // ...manejar delta
    }
    config::MQTT_TOPIC_EMERGENCY_ACTION => {
        // ...manejar emergencia
    }
    config::MQTT_TOPIC_AMR_STATUS => {
        // ...manejar AMR
    }
    config::MQTT_TOPIC_COBOT_STATUS => {
        // ...manejar cobot
    }
    _ => info!("Topic no gestionado"),
}
// Sin el '_' -> ERROR de compilacion
// si falta cualquier variante
```

**Ventaja Rust:** El compilador verifica que todos los topics conocidos tengan un handler. Agregar un topic a config.rs y olvidar el handler es un error de compilacion, no un bug silencioso en runtime.

## 6. Manejo explicito de mensajes descartados

### Cola llena: mensaje descartado

Si el sistema esta ocupado y llega un mensaje, Arduino no tiene mecanismo para detectarlo. Rust explicita el descarte con un log de error.

## Arduino / C++

*g\_comunicaciones.ino*

```
void mqttCallback(char* topic,
    byte* message, unsigned int length) {
    // Si la logica esta ocupada procesando
    // el mensaje anterior, este se acumula
    // en el stack o se pierde silenciosamente.
    //
    // No hay forma de detectar que se perdio
    // un mensaje sin instrumentacion manual.
    alRecibirMensajePorTopic(topic, msg);
}
```

## Rust

*mqtt\_manager.rs*

```
// try_send: no bloquea si la cola esta llena
if let Err(e) = event_tx.try_send(
    RobotEvent::DeltaCompleted {
        color, id_cap
    }) {
    // Descarte EXPLICITO con log de error:
    error!(
        "Cola llena - delta/status descartado: {:?}",
        e
    );
}
// El canal tiene capacidad 64:
// let (event_tx, event_rx) =
// mpsc::sync_channel:::<RobotEvent>(64);
```

**Ventaja Rust:** try\_send() devuelve error si la cola esta llena, permitiendo logearlo. Nunca bloquea el Wi-Fi stack. En Arduino el descarte es invisible.

## Resumen comparativo

| Aspecto                  | Arduino / C++                                | Rust (esp-idf-svc)                                       |
|--------------------------|----------------------------------------------|----------------------------------------------------------|
| Modelo de ejecucion      | Single-thread, polling en loop()             | Multi-core: callback Core 0, logica Core 1               |
| Reconexion               | delay(5000) bloquea TODO 15s, max 3 intentos | thread::sleep suspende solo ese hilo, reintenta infinito |
| Estado compartido        | Variables globales, sin proteccion           | Arc> garantizado por el compilador                       |
| Emergencia entre nucleos | bool global, sin garantia de visibilidad     | AtomicBool con Ordering::SeqCst, atomico por hardware    |
| Despacho de topics       | strcmp manual, fallos silenciosos            | match exhaustivo, error de compilacion si falta handler  |
| Mensajes descartados     | Silencioso, invisible                        | try_send() explicito con log de error                    |
| Decoding de payload      | Copia byte a byte en Arduino String          | Zero-copy con str::from_utf8 sobre el buffer             |
| QoS en publish           | Sin configuracion explicita                  | QoS::AtLeastOnce explicito por topic                     |
| Seguridad en compilacion | Ninguna (C++, sin borrow checker)            | Borrow checker: data races son error de compilacion      |

**Conclusion:** Arduino es adecuado para proyectos simples de un solo hilo. El problema especifico de este proyecto es que el ESP32-S3 tiene dos nucleos corriendo en paralelo, y la biblioteca PubSubClient no fue disenada para ese escenario. Rust con Arc<Mutex<>>, AtomicBool y mpsc::SyncSender es la unica forma de garantizar que el callback del broker (Core 0) y la logica del robot (Core 1) no generen data races, sin bloqueos mutuos y con verificacion en tiempo de compilacion, no en runtime.